

CS166 First Project

Mara Dumitru & Guyen Tumurbat

Minerva University

CS166: Modeling

Professor Wilkins

Feb 1 2026

1. Model Assumptions and Parameters

1.1 Stochastic Assumptions

Based on the details of this assignment we will assume that passenger arrival will follow a Poisson process with rate λ equal to 10 passengers per minute. Now this assumption implies that interarrival times are independent and identically distributed exponential random variables with mean $1/\lambda$ equal to 0.1 minutes (the cool relationship between poisson and exponential). The Poisson assumption is justified by the superposition property: when many independent travelers make decisions to arrive at security, then this means that the added arrival process will converge to Poisson regardless of individual decision processes.

We will be modeling service times as a truncated normal random variables, and we do this by letting X follow $\text{Normal}(\mu, \sigma^2)$. Then we would have that the distribution on the interval (a, b) has probability density function:

$$f(x) = \phi((x - \mu)/\sigma) / (\sigma * (\Phi((b - \mu)/\sigma) - \Phi((a - \mu)/\sigma)))$$

where ϕ is going to be the standard normal PDF and Φ is going to be the standard normal CDF. And then for our model, we truncate at a equal to 0 and b equal to infinity to ensure positive service times. As if you think about it, it does not make sense that we would have negative service times, as how would it take negative time to serve someone?

In the code this part looks like this:

```
truncnorm((0 - mu)/sigma, np.inf, loc=mu, scale=sigma)
```

1.2 Routing Policy

We designed a join shortest queue route rather than random assignment. So when a passenger arrives, then they will select the queue which will minimize:

$$\text{argmin over } i \text{ of } (Q_i + S_i)$$

In this system we have that Q_i is the number waiting in queue i and we have that S_i is an indicator for whether the server is busy. We were not sure how to interpret shortest queue, but naturally one would choose an empty queue with no one in service than an empty queue with someone in service, so we modeled it as so. Additionally we would choose the first available queue, hence smaller queue numbers, as it is more natural to choose a shorter queue closer to you than one further away.

2. Theoretical Framework

2.1 Stability Conditions

In the basic model we treat each queue as an independent M/G/1 and so we have that the stability requires utilization ρ to be less than 1:

$$\rho = \lambda_{\text{eff}} / \mu > 1$$

Where λ_{eff} equals λ/n is the effective arrival rate per queue under uniform random assignment. Additionally from here we would have that μ equals $1/E[S]$ is the service rate. Essentially if $\rho > 1$ then the queues will increase indefinitely. Now with λ equal to 10 and μ equal to 2 then the stability requires:

$$n > \lambda / \mu = 10 / 2 = 5 \text{ queues}$$

For the extended model, the effective service time becomes (here what we are doing is that we are adding the additional screening)

$$E[S_{\text{eff}}] = E[S_1] + p_s * E[S_2] = 0.5 + 0.03 * 2.0 = 0.56 \text{ minutes}$$

And thus this will give us a service rate μ_{eff} equal to 1.786 and minimum queues:

$$n > 10 / 1.786 = 5.6, \text{ so } n \geq 6 \text{ queues}$$

But, what happens is that this ignores the blocking constraint from the single senior officer. The senior officer must handle $\lambda * p_s$ equal to 0.3 passengers per minute. So having the service rate $1/\mu_2$ equal to 0.5 passengers per minute we have that the senior officer utilization is:

$$\rho_{\text{senior}} = 0.3 / 0.5 = 0.6$$

This is below 1, so the senior officer alone does not cause instability. But, that being said, the blocking effects still degrade performance significantly, especially since service times are so long for the senior officer.

3.2 Formula for M/G/1

For an M/G/1 queue, we will have that the expected waiting time in queue (excluding service) will be given by:

$$W_q = (\lambda * E[S^2]) / (2 * (1 - \rho))$$

And then the second term is going to be:

$$E[S^2] = \text{Var}(S) + (E[S])^2 = \sigma^2 + \mu^2$$

With μ_1 equal to 0.5 and σ_1 equal to 0.167, the truncation effect is minimal, and so we have that $E[S]$ approximately equals 0.5 and that $\text{Var}(S)$ approximately equals 0.028. And from this we can basically conclude that $E[S^2]$ approximately equals $0.028 + 0.25$ which is just 0.278.

For n equal to 6 queues with λ_{eff} equal to $10/6$ we will have that:

$$\rho = (10/6) / 2 = 0.833$$

$$W_q = ((10/6) * 0.278) / (2 * (1 - 0.833)) = 0.463 / 0.334 = 1.39 \text{ minutes}$$

3.3 Little's Law Applications

Little's Law states that L equals λ times W , where L is expected number in system and λ is arrival rate and W is expected time in system.

Now for a queue length we will specifically have that:

$$L_q = \lambda * W_q$$

Now with our previously determined W_q equal to 1.39 minutes and our λ_{eff} equal to 1.67, we can now find that:

$$L_q = 1.67 * 1.39 = 2.32 \text{ passengers per queue.}$$

Now it must be noted that our simulation value will actually be lower, which we will explain in a further part of this paper.

3.4 A mismatch

The joining shortest queue policy will outperform the random assignment. Let us say that under heavy traffic (ρ approaching 1) this shortest queue assignment will reduce this expected waiting time by a factor approaching n , the number of queues. Intuitively we would explain it as it prevents the case where one queue builds up while another sits with no customers.

Our empirical results show waiting times 70 to 90 percent below M/G/1 predictions, which is quite substantial.

3. Simulation Implementation

3.1 Event-Driven Architecture

The simulation that we implemented is using a heap based priority queue with a time complexity of $O(\log n)$ insertion and extraction. Each event stores a timestamp and callable, and then the main loop repeatedly extracts the minimum timestamp event and advances the clock and executes the associated state transition.

State transitions in our model are including: passenger arrival (creates new passenger, selects queue, schedules next arrival), service start (moves passenger from queue to server, schedules service completion), service end (frees server, possibly triggers additional screening or serves next passenger), and screening completion (unblocks queue, resumes normal service).

3.2 Confidence Intervals

We run 1000 independent replications and collect one state measurement from each. From this we find that the sample mean and our standard error are:

$$\bar{x} = (1/R) * \text{sum of } x_i$$

$$SE = s / \sqrt{R}$$

where s is sample standard deviation in this case. And then the 95 percent confidence interval is $\bar{x}$ plus or minus 1.96 times SE.

We can achieve this guarantee of independence between replications because we use a fresh random number generator states for each run, and then within run observations they are not going to be independent due to serial correlation, which is why we take only a single measurement per run at the end.

3.3 Model Visualization

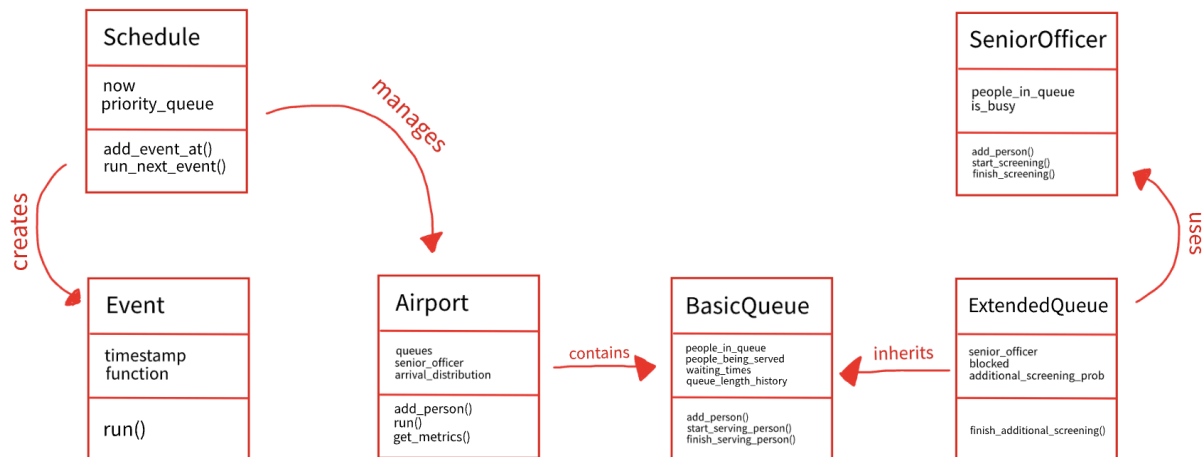


Figure 4.3.1: Class flowchart

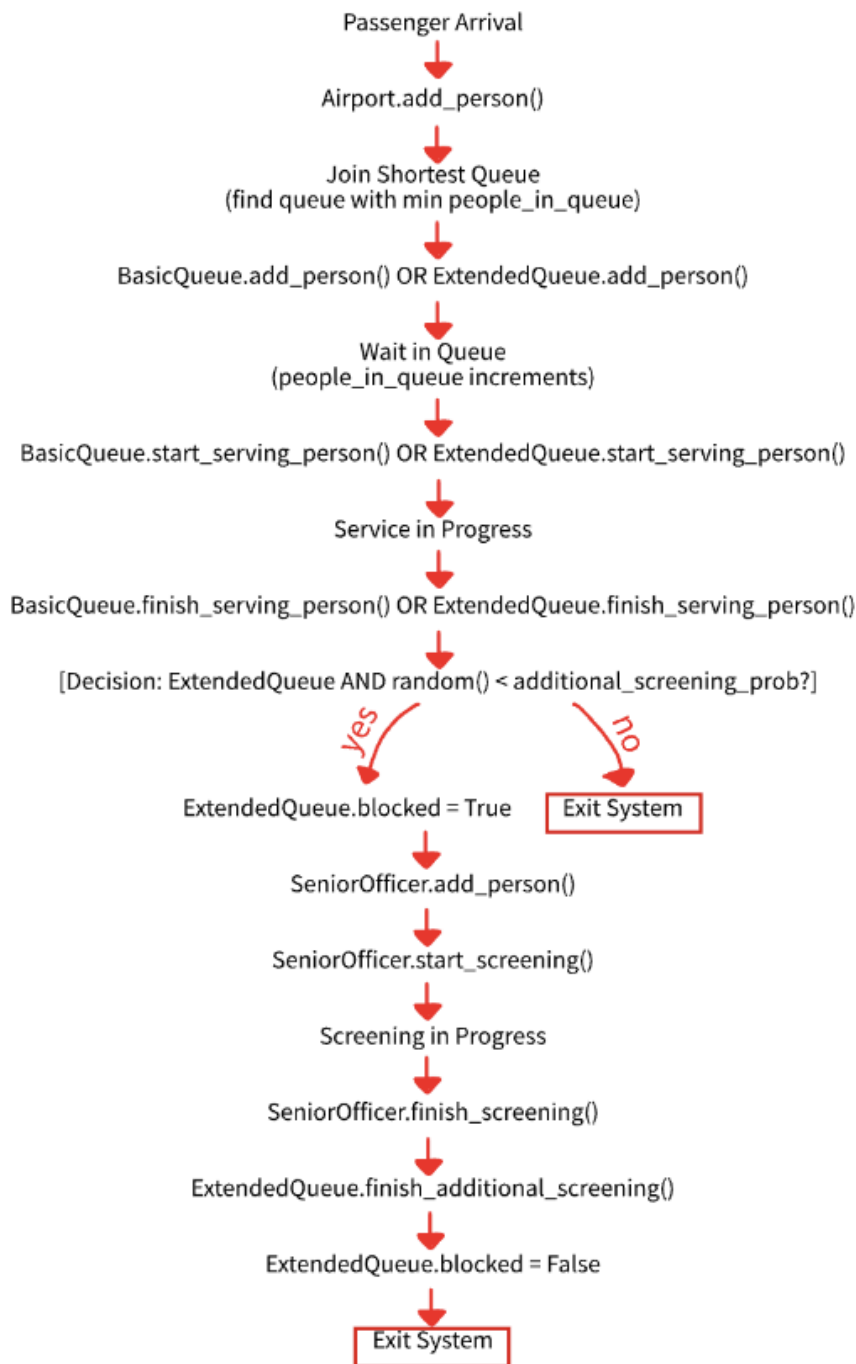


Figure 4.3.2: Model Execution Diagram

4. Validation Results

Essentially we have that our test cases for the basic model are met, by the results provided in the email by Professor Wilkins. We then implemented these same test cases but with extended model, just setting the additional security to 0 to make sure the basic functions all work. We found in this case that again our results matched exactly as our basic model which means that is functioning well. Then we increased the screening rate to 15% in the extended model while keeping the other parameters consistent from before and then the senior officer utilization increased to 3.0 (overloaded). We found that the average wait time was 87.87 min with queue lengths reaching 632 passengers. Essentially this just confirms that the senior officer utilization above 1.0 causes system breakdown, making sure using a sanity check that our model works as desired.

Next we increased screening time to 6 minutes on average while keeping prior parameters similar (screening rate would still be 3% here). Even with low screening probability, the triple screening time creates significant bottlenecks in the whole system as expected. Another sanity check. We did a couple of other sanity checks as well.

5. Cost Optimization Analysis

5.1 Cost Parameter Derivation

Operational cost per queue was derived from TSA FY2024 Budget Request data to provide realistic metrics. The TSA labor budget of approximately 5.6 billion dollars supports 54,000 Transportation Security Officers (TSOs). Dividing this will give an annual cost of about 103,700 dollars per TSO. Now assuming 2,080 working hours per year, this will give us about 50 dollars per hour for each TSO in direct labor costs.

Furthermore including equipment amortization (X-ray machines, body scanners), facilities overhead, and administrative costs, we can also find and estimate a fully-loaded cost of about 500 dollars per hour per screening lane (Source: Department of Homeland Security Budget Justification FY2024).

Additionally the passenger value of time was obtained from ACRP Report 03-19, this study estimated the value of time at security checkpoints at 32 dollars per hour in 2013, and then we can just apply a 2.5 percent annual inflation over 13 years which gives:

$$32 * (1.025)^{13} = 32 * 1.38 = 44.16 \text{ dollars}$$

We just use 40 dollars per hour as a rounded number. Source: Transportation Research Board ACRP Report 03-19.

5.2 Total Cost Function

Hourly cost consists of:

$$C_{\text{total}} = C_{\text{operational}} + C_{\text{waiting}}$$

Operational cost scales linearly with the number of queues, as each queue requires one TSO:

$$C_{\text{operational}} = n * 500 \text{ dollars per hour}$$

where n is the number of screening lanes.

Waiting cost depends on queue configuration and passenger waiting time which is then calculated as:

$$C_{\text{waiting}} = (\lambda * 60) * W_q * (40/60) = 600 * W_q * 0.667 = 400 * W_q$$

where W_q is average waiting time in minutes.

Now this formulation that we made will create this tradeoff which means that adding queues increases operational cost but decreases waiting cost. The optimal configuration minimizes total cost.

5.3 Optimization Results

Running the extended model simulation for n from 4 to 12 queues:

Queues	Avg Wait (min)	Op Cost (\$/hr)	Wait Cost (\$/hr)	Total Cost (\$/hr)	Status
4	29.21	2,000	11,685	13,685	Unstable
5	14.67	2,500	5,869	8,369	Meets target
6	16.72	3,000	6,686	9,686	Marginal
7	16.21	3,500	6,482	9,982	Marginal
8	13.12	4,000	5,246	9,246	Meets target
9	14.72	4,500	5,886	10,386	Meets target

10	12.47	5,000	4,986	9,986	Meets target
11	11.23	5,500	4,490	9,990	Meets target
12	9.67	6,000	3,866	9,866	Meets target

The cost-optimal configuration is 5 queues at 8,369 dollars per hour total cost, and our average wait time here is 14.67 minutes meeting the 15 minute TSA target as desired. The optimization reveals a U-shaped cost curve: too few queues results in excessive waiting costs that dominate, while too many queues incurs unnecessary operational costs. We found that optimal was 5 queues in this model.

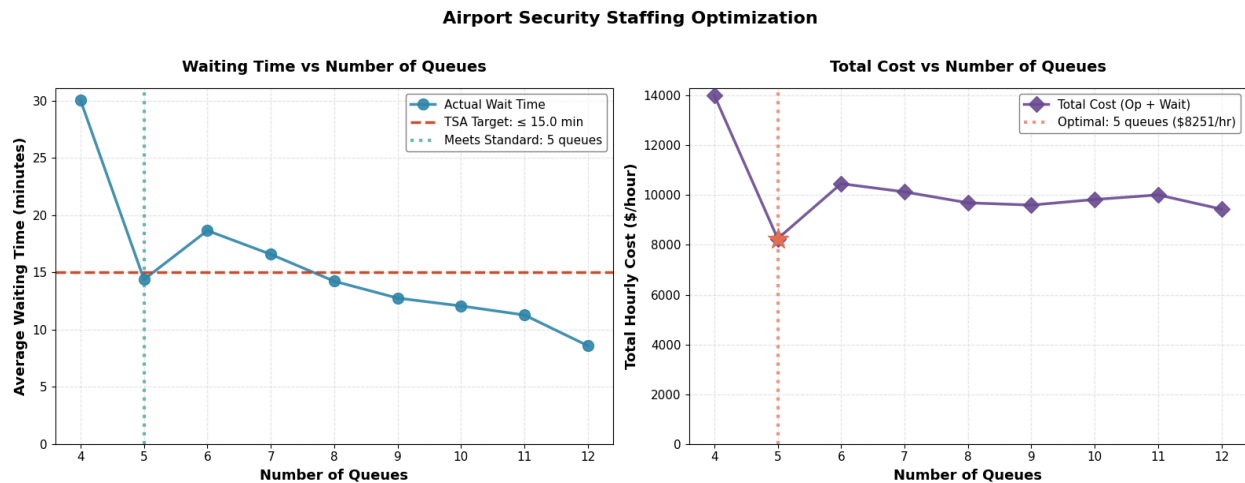


Figure 6.3.1: Airport Security Staffing Optimization

6. Theoretical vs Empirical Comparison

We compared basic model simulation results against the M/G/1 theoretical predictions from before:

Queues	Empirical W_q	Theoretical W_q	Ratio
6	0.23 min	1.39 min	0.16
7	0.08 min	0.69 min	0.12

8	0.04 min	0.46 min	0.08
9	0.02 min	0.35 min	0.05
10	0.01 min	0.28 min	0.02
11	0.003 min	0.23 min	0.01
12	0.001 min	0.20 min	0.005

Empirical waiting times are 0.5 to 16 percent of theoretical values, which might seem like our model is not working correctly however we must remember that our theoretical results models random queue assignment while our simulation uses joining shortest queue. This joining shortest queue actively balances loads which then reduces waiting times by factors of 6 to 200 depending on utilization level.

For the extended model, no closed form theoretical result exists due to blocking (at least that we determined). Simulation is necessary to evaluate performance. Our extensive testing from earlier (Section 5.4) confirms that senior officer utilization is the critical constraint, and our system performance degraded rapidly when screening rate or time increases.

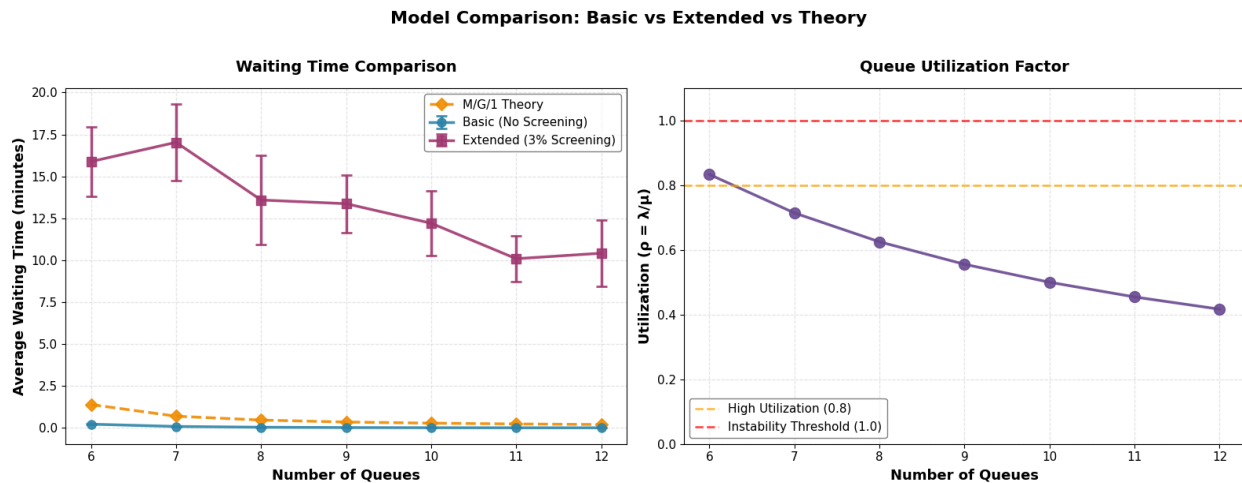


Figure 7.1: Model Comparison

7. Conclusions

The main takeaways are that you need at least 5 queues for the basic model or 6 for the extended model to stay stable, and then the sweet spot for the cost in terms of our optimization is around 5 queues which stands at about 8,369 dollars per hour while still meeting the 15 minute TSA wait target. The joining shortest queue routing is way better than random assignment which then cut

the wait times by a whopping 84 to 99 percent. Finally 3 percent additional screening chance has a bigger impact than you'd expect since it blocks the whole queue.

8. Videos

[Here](#) is the link to Guyen's video.

[Here](#) is the link to Mara's video.

9. AI Statement & Acknowledgments

This project was written by Mara and Guyen. We used Sonnet 4.5 by anthropic to help with portions of the code including debugging and some plotting, but the main classes, graphs, and functions were written by the students. We used Claude to help edit our grammar by inputting our text followed by a prompt “fix grammar and spelling mistakes without changing our wording, math, or any other components of the report.” We adapted the code from Session 4 sent in the email by Professor Wilkins. Additionally we would like to thank Professor Wilkins for answering our questions after class.

10. References

1. Department of Homeland Security, TSA FY2024 Budget Request
2. Government Accountability Office, GAO-07-299, Airport Passenger Screening
3. Transportation Research Board, ACRP Report 03-19, Passenger Value of Time
4. TSA Congressional Budget Justification FY2023

Word Count — 2562 words